Lab 04

Lab 04: Raster data management and analysis

Read the instructions COMPLETELY before starting the lab

This lab builds directly on the raster analysis concepts we worked through in class, including the Week 12 in-class notebook and the Wildcard Thursday activity. You should have that code available for reference.

Formatting your submission

This lab must be placed into a public repository on GitHub (www.github.com). Before the due date, submit **on Canvas** a link to the repository. I will then download your repositories and run your code. The code must be contained in either a .R script or a .Rmd markdown document. As I need to run your code, any data you use in the lab must be referenced using **relative path names**. Finally, answers to questions I pose in this document must also be in the repository at the time you submit your link to Canvas. They can be in a separate text file, or if you decide to use an RMarkdown document, you can answer them directly in the doc.

Data

All data for this lab are in the course repository at:

`../data/wildcard_thursday/erie_raster_ex/`

You will find two subdirectories:

- `images/` — seven weekly composite GeoTIFF rasters of cyanobacteria bloom intensity in Lake Erie (summer–fall 2017)
- `bounds/` — several shapefiles representing geographic areas of interest within the Lake Erie region

Before writing any code, read the metadata file at

`../data/wildcard_thursday/erie_raster_ex/erie_README.txt`.

This is not optional. Understanding your data source, its scaling, and its data key is essential for correct analysis.

Introduction

Cyanobacteria (blue-green algae) blooms in Lake Erie have become a significant environmental and public health concern. These blooms can produce toxins that contaminate drinking water and harm aquatic ecosystems. The data you will use were produced by NOAA using imagery from the European Space Agency's Sentinel-3 satellite. Each raster represents the maximum observed bloom intensity during a given week.

The product you will be working with is the **CIcyano** (Cyanobacteria Index, cyano only) product. Values in these rasters are stored as **digital numbers (DN)** and must be transformed to meaningful physical units before interpretation. You saw this in class — recall the reverse-scaling formula from the README and from the Week 12 notebook.

A critical aspect of working with remote sensing data is understanding **sentinel values**: special numeric codes that indicate conditions like cloud cover, land pixels, or missing data. These are NOT real observations and must be handled carefully before any analysis.

We begin by loading the relevant packages:

```r
library(terra)
```

```
## Warning: package 'terra' was built under R version 4.4.3

## terra 1.8.93
```

```r
library(sf)
```

```
## Warning: package 'sf' was built under R version 4.4.3

## Linking to GEOS 3.13.0, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE
```

```r
library(tidyverse)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3

## Warning: package 'tibble' was built under R version 4.4.3

## Warning: package 'tidyr' was built under R version 4.4.3

## Warning: package 'readr' was built under R version 4.4.3

## Warning: package 'purrr' was built under R version 4.4.3

## Warning: package 'dplyr' was built under R version 4.4.3

## Warning: package 'stringr' was built under R version 4.4.1

## Warning: package 'forcats' was built under R version 4.4.1

## Warning: package 'lubridate' was built under R version 4.4.3

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.0     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.2     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.1
```

```
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x tidyr::extract() masks terra::extract()
## x dplyr::filter()  masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(tmap)
```

```
## Warning: package 'tmap' was built under R version 4.4.1
```

```r
library(fs)
```

```
## Warning: package 'fs' was built under R version 4.4.1
```

Task 1: Data management and preprocessing

1.1 Inventory your rasters

Use `fs::dir_ls` with the parameter `regexp = "\\.tif$"` to get the file paths for all raster images in the `images/` directory. Store the result in an object called `raster_files`. Print the result — how many files are there?

```r
# your code here
```

1.2 Inspect a single raster

Load the first raster in your list using `terra::rast()`. Examine the following properties and record what you find:

- Coordinate reference system: `terra::crs()`
- Spatial extent: `terra::ext()`
- Spatial resolution: `terra::res()`
- Number of layers: `terra::nlyr()`
- Range of cell values (use `terra::global()` with `"min"` and `"max"`)

```r
# your code here
```

1.3 Write a preprocessing function: `clean_cicyano()`

Looking at the data key in the README, you will notice that values of 0 and 250 and above have special meanings — they are **not** valid bloom observations. Before doing any analysis, these sentinel values must be replaced with `NA`.

Write a function called `clean_cicyano(r)` that:

1. Takes a SpatRaster `r` as input

2. Builds a reclassification matrix that maps **0** and **all values >= 250** to `NA`, and leaves values 1–249 unchanged
3. Applies the reclassification using `terra::classify()` with the argument `others = NA` (this is a useful argument — look it up)
4. Returns the cleaned raster

Hint: Your reclassification matrix should handle the no-detection case (0) and all sentinel codes (250 through 255). Review how `terra::classify()` uses a three-column matrix to specify `[from, to, becomes]` intervals.

```
# your code here
```

Test your function on the first raster. Verify it worked by comparing the global min/max of the raw raster vs. the cleaned raster.

1.4 Write a transform function: `dn_to_ci()`

The DN values in these rasters are scaled according to the formula in the README. To convert back to physical CI units, use the reverse-scaling formula:

$$CI = 10^{\left(\frac{3.0}{250.0} \times DN - 4.2\right)}$$

Write a function called `dn_to_ci(r)` that takes a cleaned SpatRaster and returns a new raster with values transformed to CI units. You can apply a function to a raster using standard arithmetic operations or by passing a function to the raster.

```
# your code here
```

Test your function. What is the range of CI values in the transformed raster? Do they make physical sense given the data key?

Questions for Task 1:

Q1.1 What does the CRS of these rasters tell you about how distances and areas are measured? Would you need to reproject before calculating bloom area in square kilometers?

Q1.2 Why is it important to remove sentinel values *before* applying the DN-to-CI transform — rather than after? What would happen to a land pixel (value = 252) if you transformed it without cleaning first?

Q1.3 The `others = NA` argument in `terra::classify()` is useful here. In your own words, what does it do?

Task 2: Spatial subsetting — crop and mask

2.1 Load and align an area of interest

Load the `bb_erie_sandusky` shapefile from the `bounds/` directory using `sf::read_sf()`. Check its CRS. If it does not match the raster CRS, reproject it using `sf::st_transform()`.

```
# your code here
```

2.2 Crop and mask

Using the **August 16–22, 2017** raster (`ts_2017.0816_0822`):

1. Apply `clean_cicyano()` and `dn_to_ci()` to get a clean, transformed raster
2. Use `terra::crop()` to clip the raster extent to the Sandusky Bay boundary
3. Use `terra::mask()` to set pixels *outside* the boundary to `NA`

Store your final result as `sandusky_0816`.

```
# your code here
```

2.3 Compare full vs. masked

Using `terra::global()`, calculate and compare the **mean** CI value for:

- The full Lake Erie cleaned/transformed raster
- The Sandusky Bay-masked raster (`sandusky_0816`)

```
# your code here
```

2.4 Map it

Create a `tmap` map of `sandusky_0816`. Your map should include:

- A meaningful title
- A legend with appropriate labels
- A scale bar

```
# your code here
```

Questions for Task 2:

Q2.1 In your own words, what is the difference between `terra::crop()` and `terra::mask()`? Why did we use both here instead of just one?

Q2.2 The mean CI value for Sandusky Bay is likely different from the full lake mean. What could explain this? Does this necessarily mean Sandusky Bay has more or less cyanobacteria than average?

Task 3: Multi-temporal comparison

3.1 Load and stack multiple rasters

Load and clean **at least four** of the seven rasters (choose dates that span a meaningful time range). Apply `clean_cicyano()` and `dn_to_ci()` to each. Then combine them into a single multi-layer SpatRaster using `c()`.

Tip: using `purrr::map()` over your `raster_files` vector, combined with your preprocessing functions, can make this more efficient.

```
# your code here
```

3.2 Temporal summary

Use `terra::global()` on your multi-layer raster to calculate the **mean** CI value per layer (use `na.rm = TRUE`). This will give you one mean value per date.

Organize the results into a data frame with at minimum two columns: `date` and `mean_ci`. You will need to extract the date from each filename. Use `basename()` and/or `stringr::str_extract()` to parse the date string from the file path — the date range is the part of the filename that looks like `0802_0808`.

```
# your code here
```

3.3 Plot the time series

Using `ggplot2`, create a line or point plot showing mean CI values over time. Label your axes clearly. Which date in your dataset shows the highest mean bloom intensity?

```
# your code here
```

3.4 Binary bloom comparison

Choose any **two dates** from your dataset. For each, create a binary raster where cells with valid CI values are coded as `1` and all other cells (NA) as `0`. You will need to think about how to do this using `terra::classify()` or logical raster operations.

Use these binary rasters to create two new rasters:

- **Persistent bloom:** cells where blooms were present on *both* dates
- **New bloom:** cells where blooms were present on the *later* date but NOT the earlier date

Map both results.

This requires careful logical thinking before writing code. Plan your approach in pseudocode first.

```
# your code here
```

Questions for Task 3:

Q3.1 What are the limitations of using **mean CI value** as a measure of bloom severity across the whole lake? What would be a more robust alternative metric?

Q3.2 Stacking rasters with `c()` in terra creates a multi-layer SpatRaster. What requirement must all rasters meet in order to be stacked this way? (Think: extent, resolution, CRS.)

Task 4: Zonal analysis with areas of interest

4.1 — Load all boundary files

Use `list.files()` to get all `.shp` files from the `bounds/` directory. Load each with `sf::read_sf()`, then combine them into a single sf object using `dplyr::bind_rows()`. Make sure each polygon has an identifier — if not, add a column with the filename or AOI name.

```
# your code here
```

4.2 Extract by area

Using the date you identified in Task 3.3 as having the highest mean bloom intensity, extract CI values for each area of interest using `terra::extract()`. Calculate the **mean** and **maximum** CI per AOI.

Tip: `terra::extract()` returns a data frame. The `ID` column links rows to the AOI that was used for extraction. You can use `group_by()` and `summarize()` to aggregate.

```
# your code here
```

4.3 Summary table

Present your results as a clean summary table. Which AOI had the highest mean CI value on that date? Which had the highest maximum?

```
# your code here
```

4.4 Map with AOI overlays

Create a `tmap` map showing the CI raster for your chosen date with the AOI boundaries overlaid. Use `tm_borders()` for the boundaries and label them if possible.

```
# your code here
```

Questions for Task 4:

Q4.1 You may have gotten `NA` values in your `terra::extract()` results for some AOIs. What could cause this? Name at least two reasons.

Q4.2 Looking at the spatial distribution of your AOIs on the map, which areas do you expect to be most prone to severe blooms, and why? (Consider geography, hydrology, land use in the Lake Erie watershed.)

Questions

1. Throughout this lab you have been working with **raster** data. In what ways was the analysis workflow different from working with vector data (like you did in Labs 1–3)? What concepts translated directly, and what was fundamentally different?

2. Cyanobacteria blooms are affected by temperature, nutrient runoff, and lake circulation. Based on the temporal patterns you observed in Task 3, does the timing of peak bloom match what you would expect ecologically? Explain your reasoning (I know you're not an expert on blooms - just demonstrate thinking).

3. We used `terra::extract()` in Task 4 to perform a zonal summary. How does this relate conceptually to the **zonal operations** category of map algebra discussed in class?